Triggers

In this section, we are going to understand the working of the **Triggers, why we need to use the triggers** and when to use them and also see the **merits and demerits** of **triggers, features of Triggers** and various commands, which are performed under the Trigger section.

What are Triggers?

A trigger is a special user-defined function connected with a table. if we want to generate a new trigger:

- Firstly, we can specify a trigger function.
- Secondly, bind the same trigger function to a table.

Note: The major variance between a trigger and a user-defined function is that, when any triggering event occurs, a trigger is automatically raised.

A Trigger is a function, which involved automatically whenever an event linked with a table. The event can be described as any of the following INSERT, UPDATE, DELETE or TRUNCATE.

Type of Triggers

In PostgreSQL, the trigger can be categorized into two parts, which are as follows:

- **Row-level trigger**
- **Statement-level trigger**

For example, if we issue an **UPDATE** command, which affects 10 rows, the **row-level trigger** will be invoked **10 times**, on the other hand, the **statement level trigger** will be invoked **1 time**.

How can Trigger be used ?

- A trigger can also be marked with the **FOR EACH** command operator through its creation, and the same trigger can be implemented only once for a particular operation.
- We can use the trigger with the **FOR EACH ROW** operator in its creation, and those triggers will be called once for each row changed by the operation.

Usage of Triggers

The triggers can be used in the following aspects:

- The triggers can be used to authenticate the input data.
- The triggers can also implement business rules.
- It can easily retrieve the system functions.
- The triggers can be used to create a unique value for a newly-inserted row in a diverse file.
- By using trigger, we can repeat the data in a different file to reach data reliability.
- It is used to write to add files to fulfil the objective of the audit trail.
- The trigger can be used to get the data from other files for cross-referencing objectives.

The Various command used in PostgreSQL Triggers

In PostgreSQL trigger, we can perform the following commands:

- **CREATE Trigger**
- **ALTER Trigger**
- **DROP Trigger**
- **ENABLE Trigger**
- **DISABLE Trigger**

Let us understand them one by one:

- **Create trigger:** SQL, the CREATE TRIGGER command generates our first trigger step by step.
- **Alter trigger:** The ALTER TRIGGER command is used to rename a trigger.
- **Drop trigger:** The DROP TRIGGER command is used to define the steps to remove a trigger from a table
- **Enable triggers:** In the PostgreSQL trigger, the ENABLE TRIGGER statement allows a trigger or all triggers related to a table.
- **Disable trigger:** The DISABLE TRIGGER is used to display how we can disable a trigger or all triggers linked with a table.

Features of PostgreSQL Triggers

Some of the essential features of the PostgreSQL triggers are as follows:

- PostgreSQL will execute the trigger for the TRUNCATE event.
- PostgreSQL allows us to specify the statement-level trigger on views.
- PostgreSQL needs to specify a user-defined function as the action of the trigger, whereas the SQL standard provides us to use any SQL commands.

PostgreSQL Create Trigger

In this section, we are going to understand the working of the **Trigger function, the creation of trigger function, PostgreSQL Create Trigger**, and the **examples** of the **Create Trigger command**.

What is the Trigger function?

A trigger function is parallel to the consistent user-defined function. But a trigger function can return a value with the type trigger and does not take any parameters.

Syntax of Create trigger function

The syntax for creating a trigger function is as follows:

```
CREATE FUNCTION trigger_function()
   RETURNS TRIGGER
   LANGUAGE PLPGSQL
AS $$
BEGIN
   -- trigger logic goes here?
END;
$$
```

Note: We can generate a trigger function with the help of any supported languages through PostgreSQL.

CREATE FUNCTION

CREATE FUNCTION trigger_function()

- CREATE FUNCTION is the keyword to create a new function.
- trigger_function is the name of the function.

RETURNS TRIGGER

RETURNS TRIGGER

- RETURNS specifies the return type of the function.
- TRIGGER is a special return type that indicates the function is a trigger function.

LANGUAGE PLPGSQL

- LANGUAGE specifies the language in which the function is written.
- PLPGSQL is the language, which is a procedural language that is similar to SQL.

AS $$

- AS is a keyword that separates the function declaration from the function body.
- $$ is a delimiter that marks the beginning of the function body. The function body is enclosed in a pair of $$ delimiters.

Function Body

```
BEGIN

  -- trigger logic goes here?

END;

$$
```

- **BEGIN** marks the start of the function body.
- -- trigger logic goes here? is a comment indicating where you should put the trigger logic.
- **END;** marks the end of the function body.
- The final $$ delimiter marks the end of the function body.

A **trigger function** can accept the data about its calling environment over a special structure called **Trigger Data** that holds a set of local variables.

For example, before or after the triggering event the **OLD and NEW** signify the row's states in the table.

PostgreSQL also allows us to other local variables preceded by **TG_** like, as **TG_WHEN**, and **TG_TABLE_NAME.**

If we specify a trigger function, we can fix the various trigger events, for example, INSERT, DELETE and Update.

How to Create a New Trigger

We will follow the below process to generate a new trigger in PostgreSQL:

Step1: Firstly, we will create a trigger function with the help of the CREATE FUNCTION command.

Step2: Then, we will fix the trigger function to a table with the help of the CREATE TRIGGER command.

What is the CREATE TRIGGER command?

The CREATE TRIGGER command is used to create a new trigger.

Syntax of PostgreSQL CREATE TRIGGER command:

The syntax of the PostgreSQL **CREATE TRIGGER** command is as follows:

```
CREATE TRIGGER trigger_name
```

```
{BEFORE | AFTER} { event }

ON table_name

[FOR [EACH] { ROW | STATEMENT }]

EXECUTE PROCEDURE trigger_function
```

In the above syntax, we have used the following parameters, as shown in the below table:

Parameters	**Description**
Trigger_name	It is used to define the trigger name after the **TRIGGER** keyword.
BEFORE|AFTER	These parameters are used when we need to define the timing at the trigger's execution, and it can be specified as **AFTER or BEFORE** when an event occurs.
Event	The event parameter is used to define the event which requested the trigger, and it can be **INSERT, UPDATE, DELETE, or TRUNCATE.**

Table_name	The table_name parameter is used to define the table name, which is linked with the trigger. And it is specified after the **ON** keyword.
[FOR [EACH] { ROW | STATEMENT}]	These parameters can define the types of the trigger, which are **Row-level trigger** and **Statement Level trigger.** ○ The **FOR EACH ROW** clause is used to define the Row-Level Trigger. ○ And the **FOR EACH STATEMENT** clause is used to specify the Statement-Level trigger.
Trigger_function	It is used to define the trigger function name after the **EXECUTE PROCEDURE** keyword.

For example, let's assume a table that has 50 rows and two triggers which will be executed when a **DELETE** event happens.

If the **delete command** removes 50 rows, the **row-level trigger** will be implemented 50 times, once for each deleted row. However, a **statement-level trigger** will be executed for one time irrespective of how many rows are removed.

Example of PostgreSQL Create Trigger

Let us see a sample example to understand the working of the **PostgreSQL CREATE Trigger** command.

We are **creating one new table** as ***Clients*** with the **CREATE** command's help and inserting some values using the INSERT command.

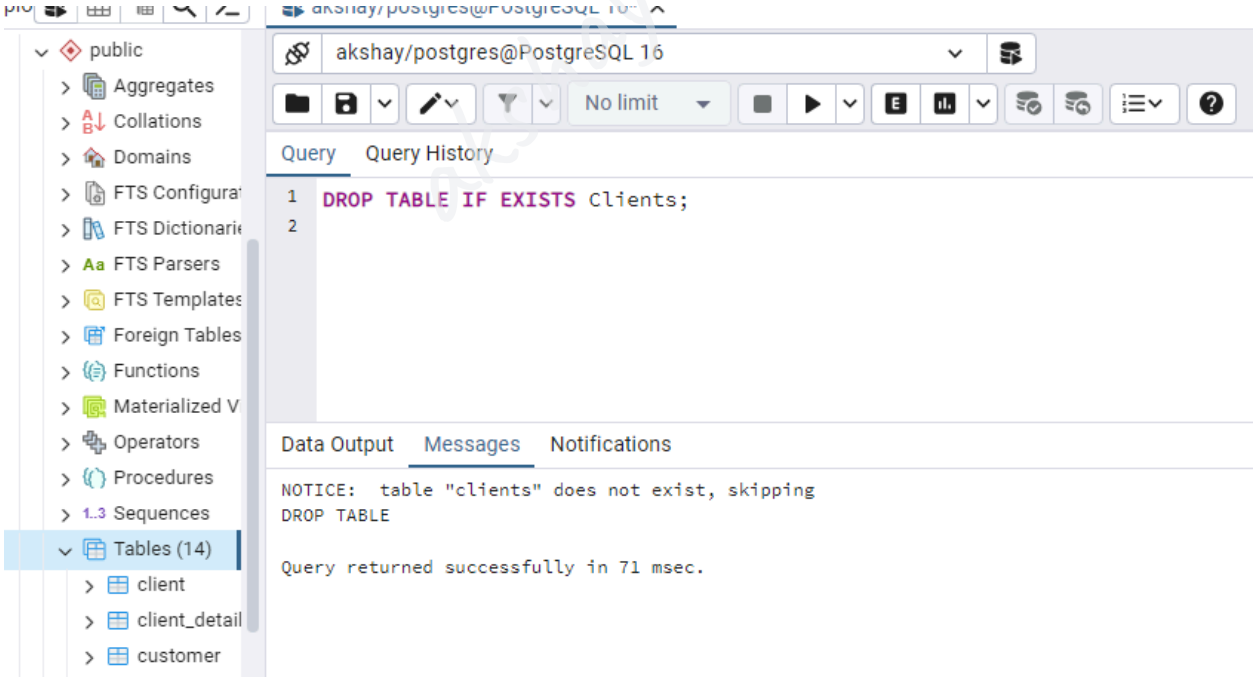
To create ***Clients*** into an **Organization database**, we use the **CREATE** command.

But, before creating the ***Clients*** table, we will use the **DROP TABLE** command if a similar table is already existing in the **Organization** database.

```
DROP TABLE IF EXISTS Clients;
```

Output

After executing the above command, we will get the following window message: the ***Clients*** table does not exist.



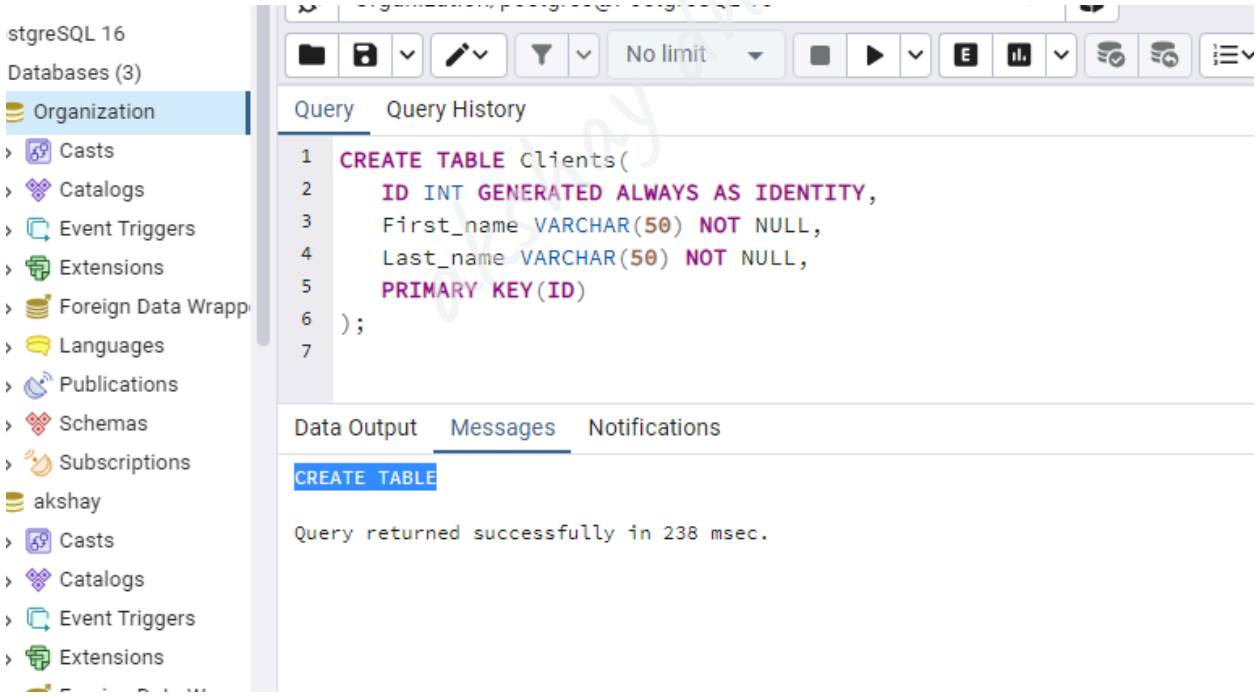
The ***Clients*** table contains various columns such as **Client_id, First_name, Last_name** column, where we use the **Client_id** as the **GENERATED ALWAYS AS IDENTITY constraint**.

```
CREATE TABLE Clients(
   ID INT GENERATED ALWAYS AS IDENTITY,
   First_name VARCHAR(50) NOT NULL,
   Last_name VARCHAR(50) NOT NULL,
   PRIMARY KEY(ID)
);
```

Output

GENERATED ALWAYS AS IDENTITY **clause is used to create an auto-incrementing column, also known as a serial column.**

On executing the above command, we will get the following message, which displays that the ***Clients*** table has been created successfully into the **Organization** database.

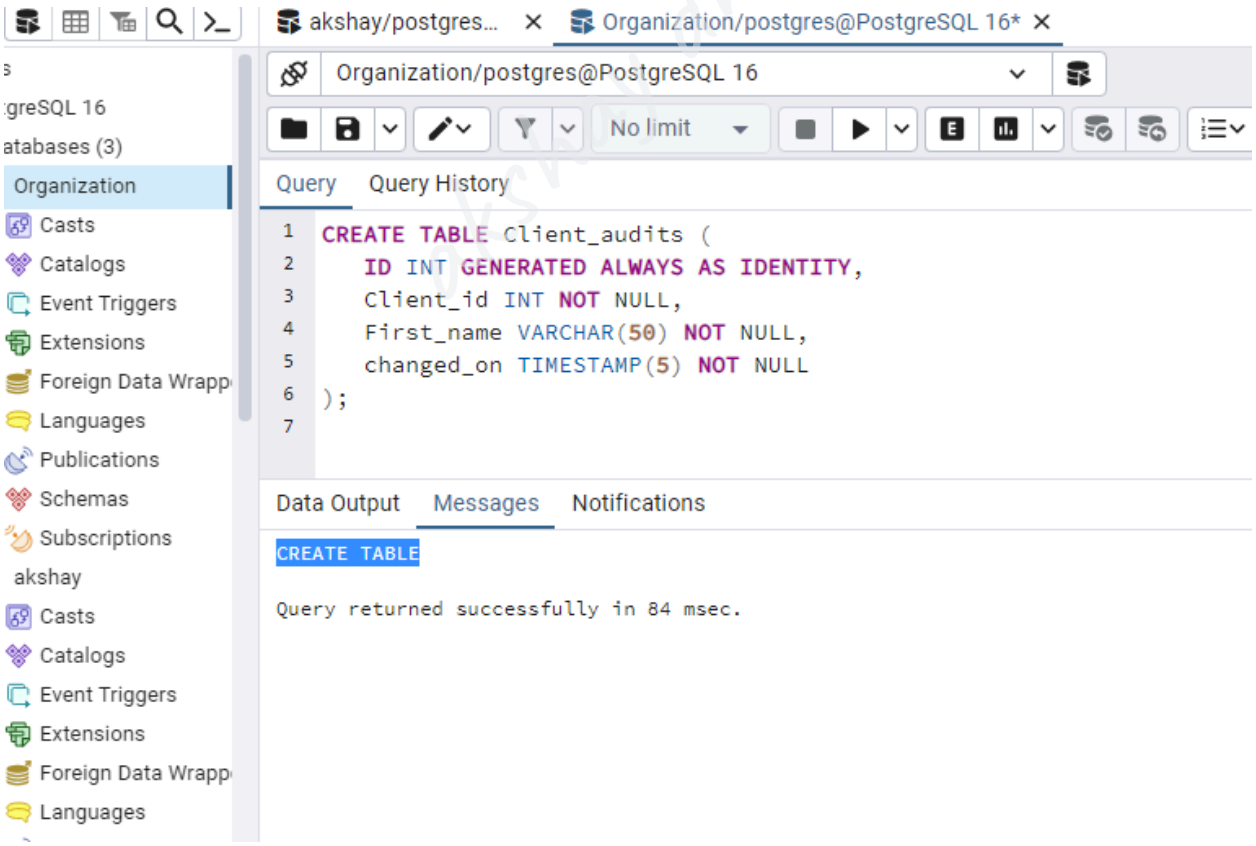


Assume that when the name of clients modifies, we want to log the modification in a different table called **Client_audits**:

```
CREATE TABLE Client_audits (
   ID INT GENERATED ALWAYS AS IDENTITY,
   Client_id INT NOT NULL,
   First_name VARCHAR(50) NOT NULL,
   changed_on TIMESTAMP(5) NOT NULL
);
```

Output

After implementing the above command, we will get the following message window, which displays that the ***Client_audits*** table has been created successfully into the ***Organization*** table.



Now, we will be following the below steps to create a new function for the specified table:

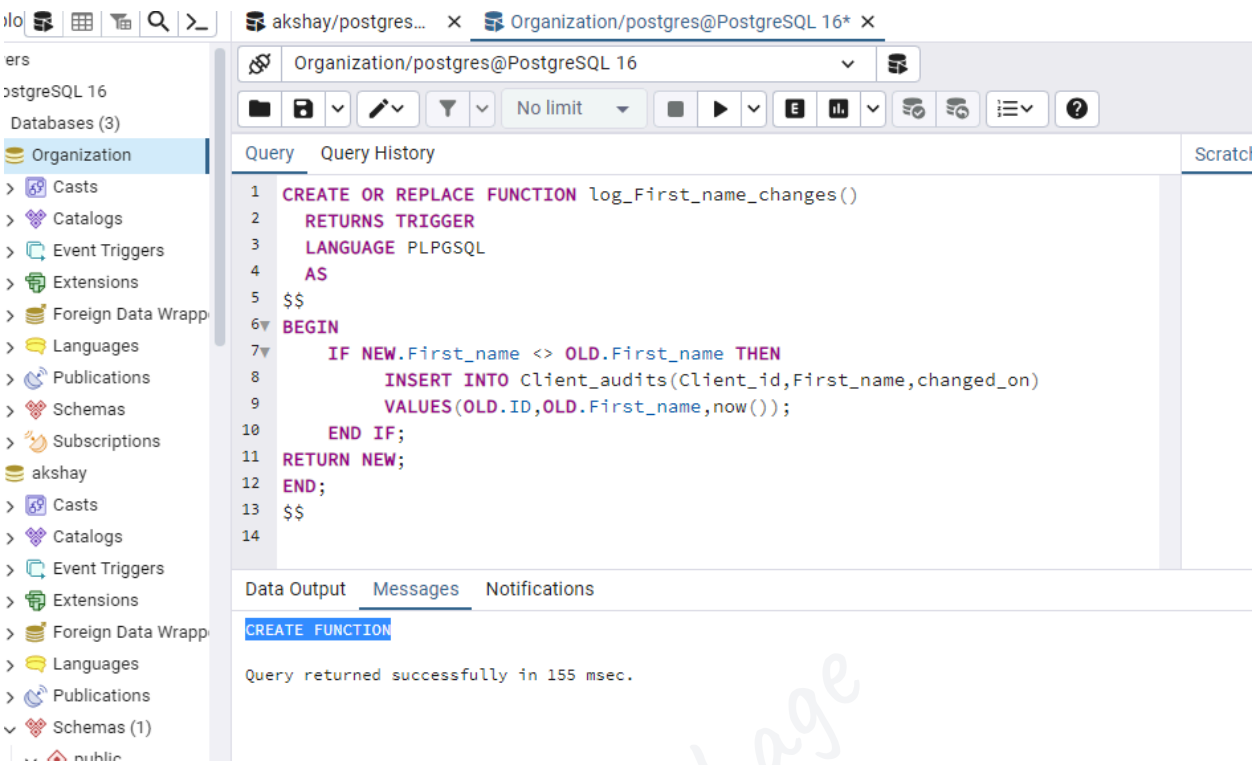
Step1: Creating a new Function

Firstly, we are creating a new function called **log_First_name_changes** using the below command:

```
CREATE OR REPLACE FUNCTION log_First_name_changes()
  RETURNS TRIGGER
  LANGUAGE PLPGSQL
  AS
$$
BEGIN
    IF NEW.First_name <> OLD.First_name THEN
        INSERT INTO Client_audits(Client_id,First_name,changed_on)
        VALUES(OLD.ID,OLD.First_name,now());
    END IF;
RETURN NEW;
END;
$$
```


Output

After implementing the above command, we will get the below message window displaying that the **log_First_name_changes** function has been created successfully into a similar database.



The function inserts the old First name into the **Client_audits** table, which contains **Client_id, First_name**, and the **time of change** if the **First_name** of a client.

In the above command, we have the following:

- The **NEW**denotes **the new row that will be updated,** whereas the **OLD** signifies **the row before the update.**
- The **First_name**retrieves **the new last name;** on the other hand, the **OLD.first_name** retrieves **the first name before the update**.

Step2: Creating a new Trigger

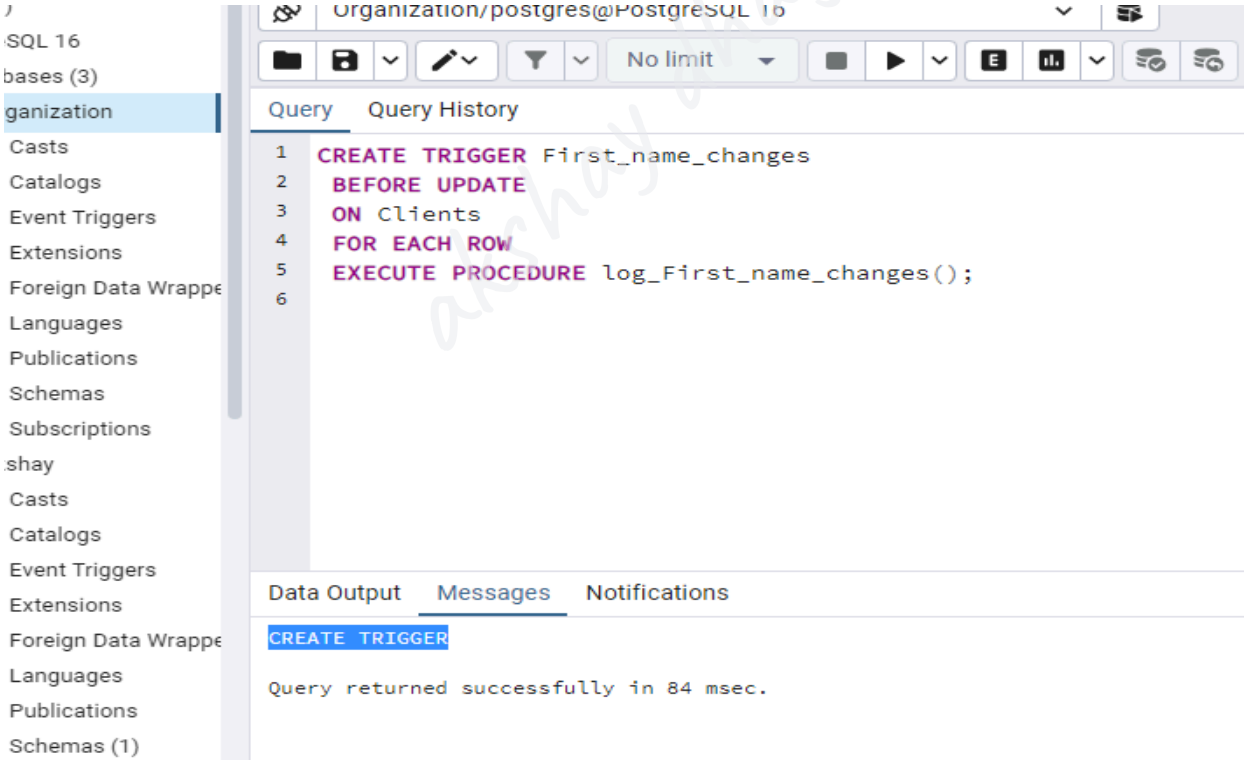
After creating a new function (**log_First_name_changes)** successfully, we will fix the trigger function to the **Clients** table where the trigger_name is **First_name_changes**.

The trigger function is used to log the modification automatically before the value of the **First_name** column is updated, as shown in the following command:

```
CREATE TRIGGER First_name_changes
 BEFORE UPDATE
 ON Clients
 FOR EACH ROW
 EXECUTE PROCEDURE log_First_name_changes();
```

Output

We will get the following message on executing the above command, which displays that the **First_name_changes** trigger has been created successfully.



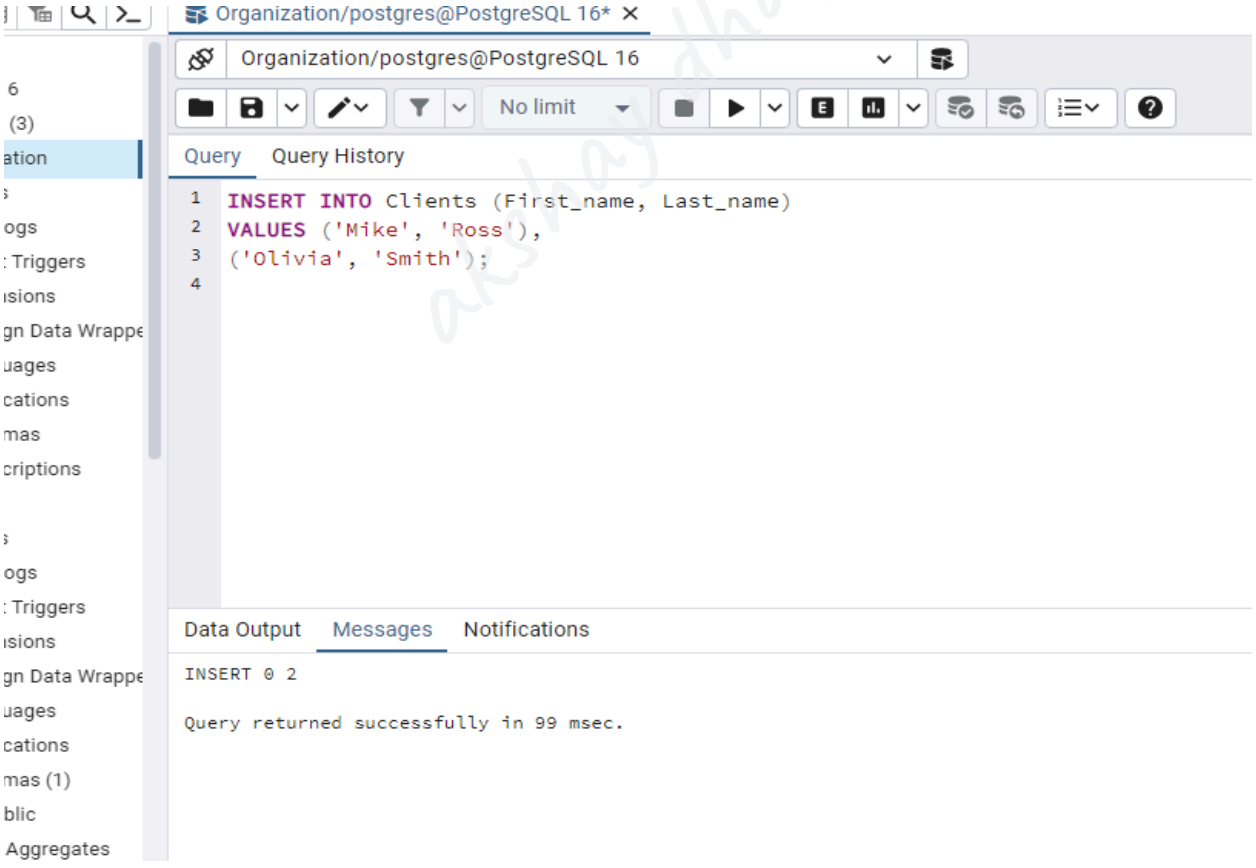
Step3: Inserting the Data

After creating the **new function as *log_First_name_changes()*** and **new trigger** as ***First_name_changes*** successfully, we will enter some values into the ***Clients*** table with the **INSERT** command's help.

```
INSERT INTO Clients (First_name, Last_name)

VALUES ('Mike', 'Ross'),

('Olivia', 'Smith');
```

Output

After implementing the above command, we will get the following message window, which displays that the two values have been inserted successfully into the ***Clients*** table.



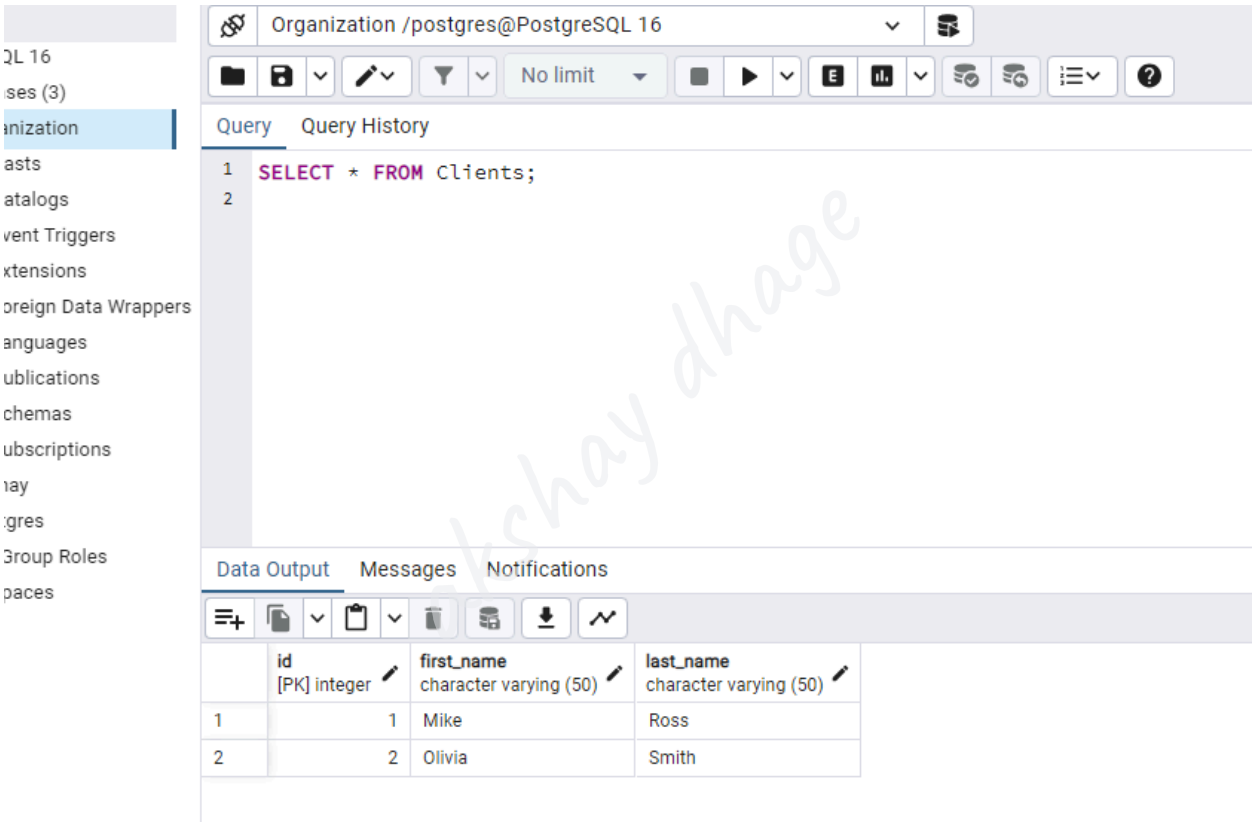
Step4: Retrieving the data

After creating and inserting the ***Clients*** table's values, we will use the **SELECT** command to retrieving the data from the ***Clients*** table:

```
SELECT * FROM Clients;
```

Output

After successfully implementing the above command, we will get the below result, which displays that the PostgreSQL returns the data present in the ***Clients*** table:



Assume that **Olivia Smith** modified her **First_name** to **Alivia Smith**.

Step5: Updating the First_name

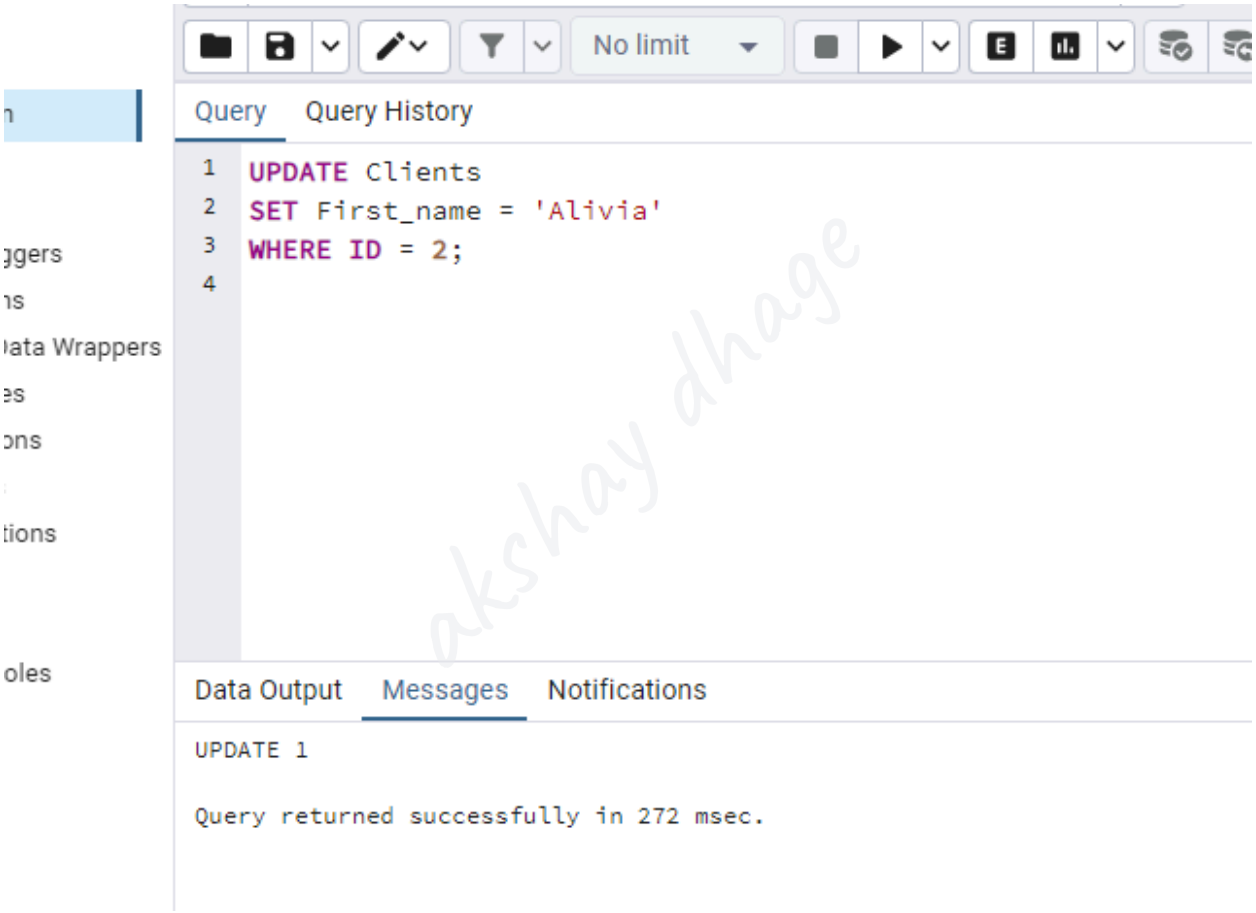
So here, we are updating Olivia's **first name** to the new one with the help of the **UPDATE** command, as shown below:

```
UPDATE Clients
```

```
SET First_name = 'Alivia'

WHERE ID = 2;
```

Output

On implementing the above command, we will get the following window message, which displays that the specified value have been updated successfully.



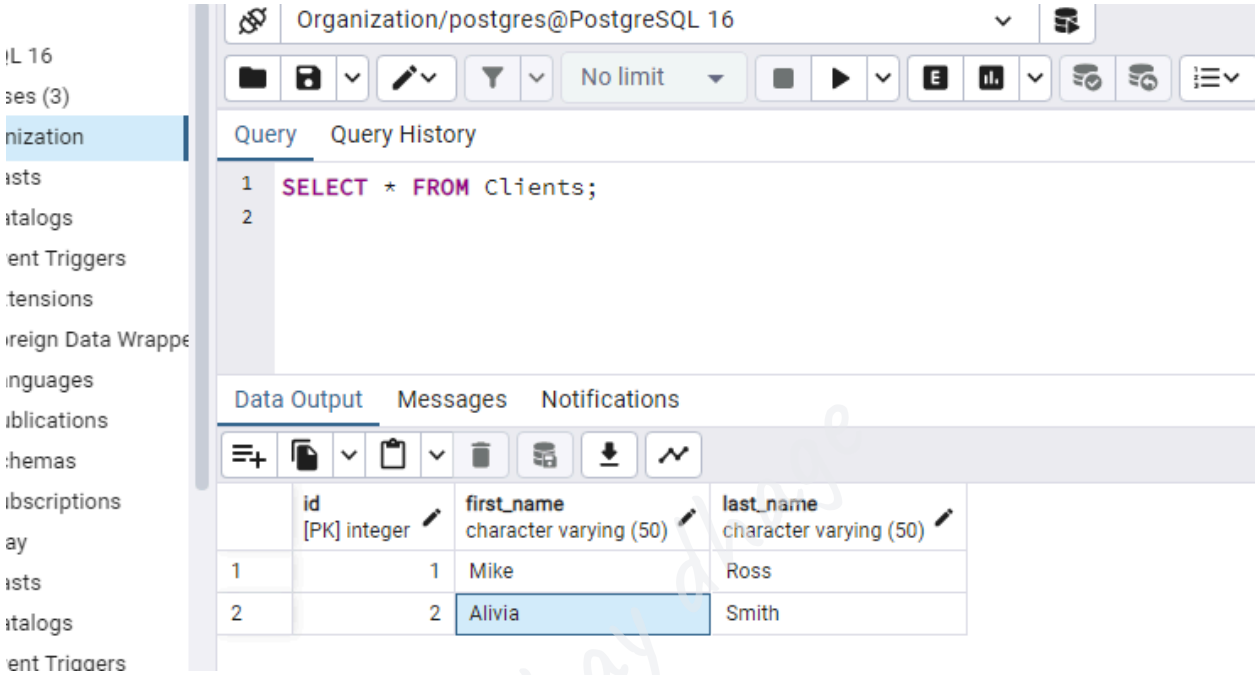
Step7: Verifying the Data after modification

Now, we will verify that if the **First name** of **Olivia** has been updated successfully or not using the following **SELECT** command:

```
SELECT * FROM Clients;
```

Output

After successfully implementing the above command, we will get the below output: Olivia's first name has been updated to **Alivia** into the ***Clients*** table:



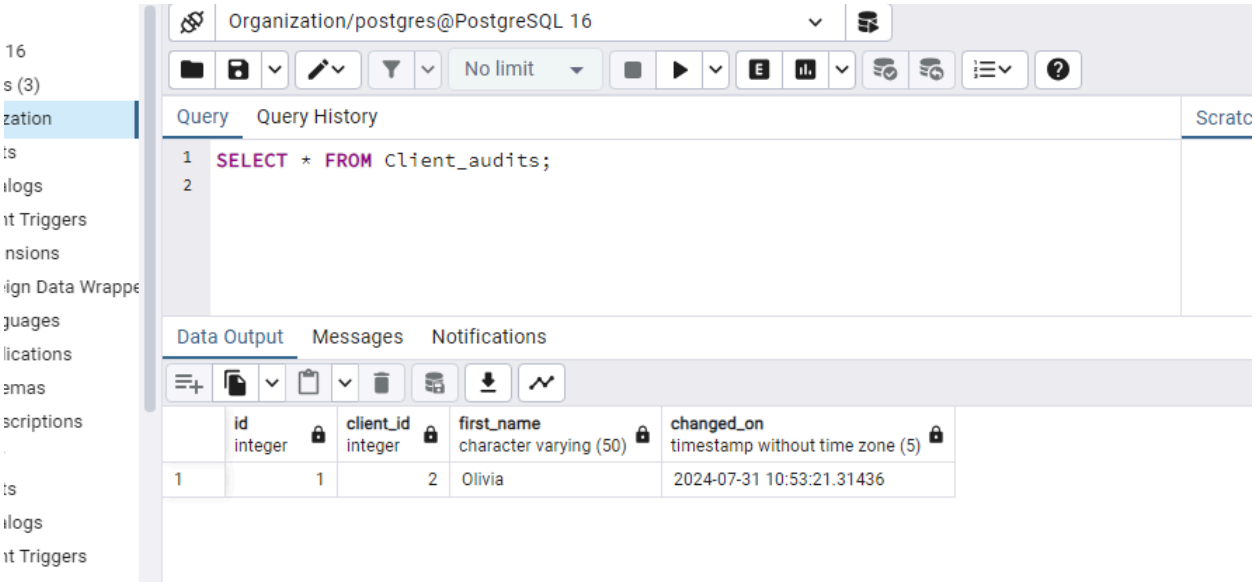
Step8: Validate the contents

After performing all the above steps successfully, in the end, we will validate the contents of the ***Client_audits*** table with the help of the following **SELECT** command:

```
SELECT * FROM Client_audits;
```

Output

After executing the above command, we will get the following output, which displays that the modification was logged in the ***Client_audits*** table by the trigger.



SQL DROP TRIGGER

In this section, we are going to understand the working of the **PostgreSQL DROP TRIGGER** command and see the example of **dropping and deleting** a trigger from a specified table in PostgreSQL.

What is the Drop Trigger command?

In PostgreSQL, we can use the **Drop Trigger** command to remove the existing trigger.

The syntax of the PostgreSQL Drop trigger command

The following illustration is used to drop a trigger from a particular table:

```
DROP TRIGGER [IF EXISTS] trigger_name

ON table_name [ CASCADE | RESTRICT ];
```


In the above syntax, we have used the following parameters:

Parameters	**Description**
Trigger_name	○ It is used to define the trigger name that we need to remove, and it is mentioned after the **DROP TRIGGER** keyword.

If EXISTS	○ The If EXISTS parameter is used to remove the trigger temporarily only if it exists. ○ And if we try to remove a **non-existing** trigger without specifying the **IF EXISTS**command, we will get an error in the result. ○ PostgreSQL issues a notice as an alternative if we use the **IF EXISTS** to remove a non-existing trigger.
Table_name	○ The **table name** parameter is used to define the table name where the trigger belongs. ○ If the table is linked to a defined schema, we can use the table's schema-qualified name, such as **schema_name.table_name.**
CASCADE	○ If we want to drop objects, which automatically rely on the trigger, we can use the **CASCADE** option.
RESTRICT	○ We can use the **RESTRICT** option if any objects depend on trigger or we want to refuse or drop that trigger. ○ The **DROP TRIGGER** command uses the RESTRICT option by default.

DROP TRIGGER trigger_name;

Example of Drop Trigger command

Let us see a simple example to understand the working of the **DROP Trigger** command.

For this, we are taking the ***Employee*** table, which we created in the earlier section of the SQL tutorial.

Step1: Creating a new function

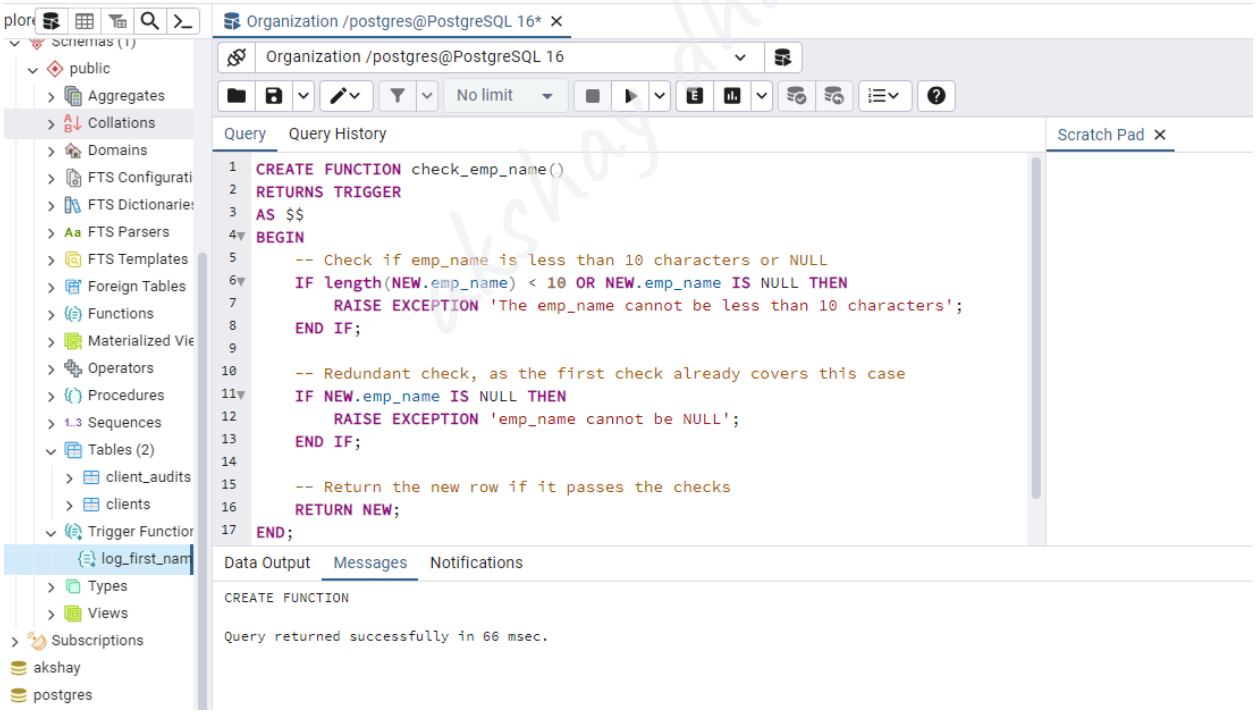
Firstly, we will create a function, which checks the **employee's emp_name**, where the **name of the employee** length must be at least **10** and must not be null.(use database akshay)

```
CREATE FUNCTION check_emp_name()
RETURNS TRIGGER
AS $$
BEGIN
    -- Check if emp_name is less than 10 characters or NULL
    IF length(NEW.emp_name) < 10 OR NEW.emp_name IS NULL THEN
        RAISE EXCEPTION 'The emp_name cannot be less than 10 characters';
    END IF;
    -- Redundant check, as the first check already covers this case
    IF NEW.emp_name IS NULL THEN
        RAISE EXCEPTION 'emp_name cannot be NULL';
    END IF;
```

```
    -- Return the new row if it passes the checks
    RETURN NEW;
END;
$$
LANGUAGE plpgsql;
```

Output

On executing the above command, we will get the following message: the **check_emp_name()** function has been created successfully into the **Organization** database.



Step2: Creating a new Trigger

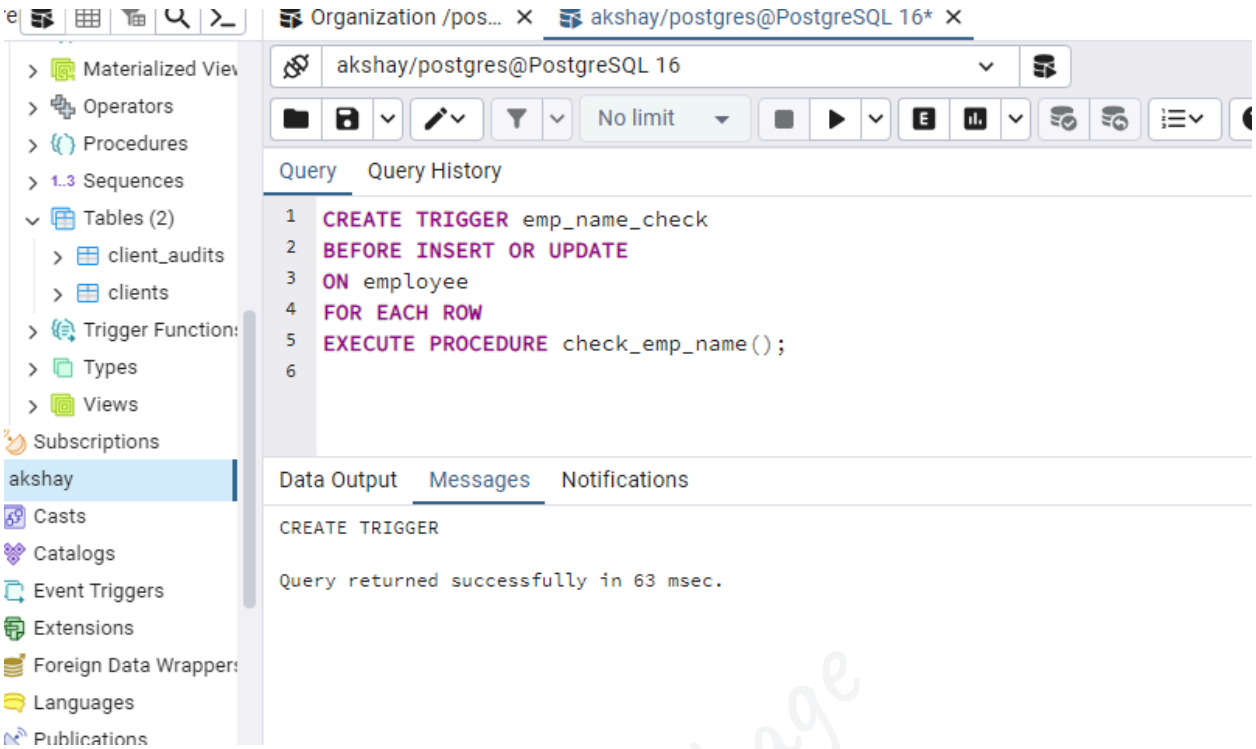
After creating the **check_emp_name()** function, we will **create a new trigger** on the ***employee*** table to check an **employee's emp_name.**

And the same trigger will be executed whenever we update or insert a row in the ***Employee*** table (taken from the **Organization** database):

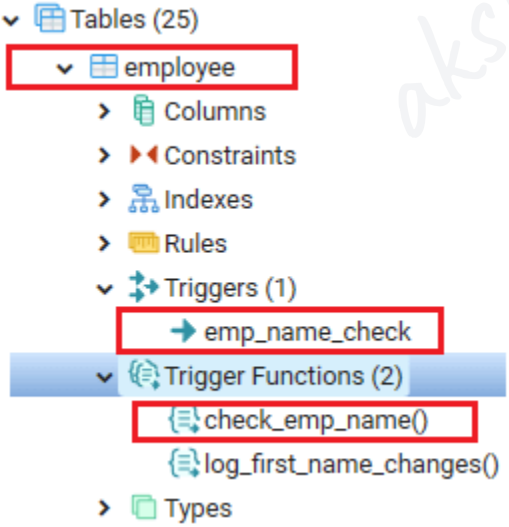
```
CREATE TRIGGER emp_name_check

BEFORE INSERT OR UPDATE

ON employee

FOR EACH ROW

EXECUTE PROCEDURE check_emp_name();
```

Output

After implementing the above command, we will get the following message window, which displays that the **emp_name_check** trigger has been inserted successfully for the ***Employee*** table.



And, we can also verify that the above created function**(check_emp_name())** and trigger**(emp_name_check**) in the object tree o**f akshay** database.



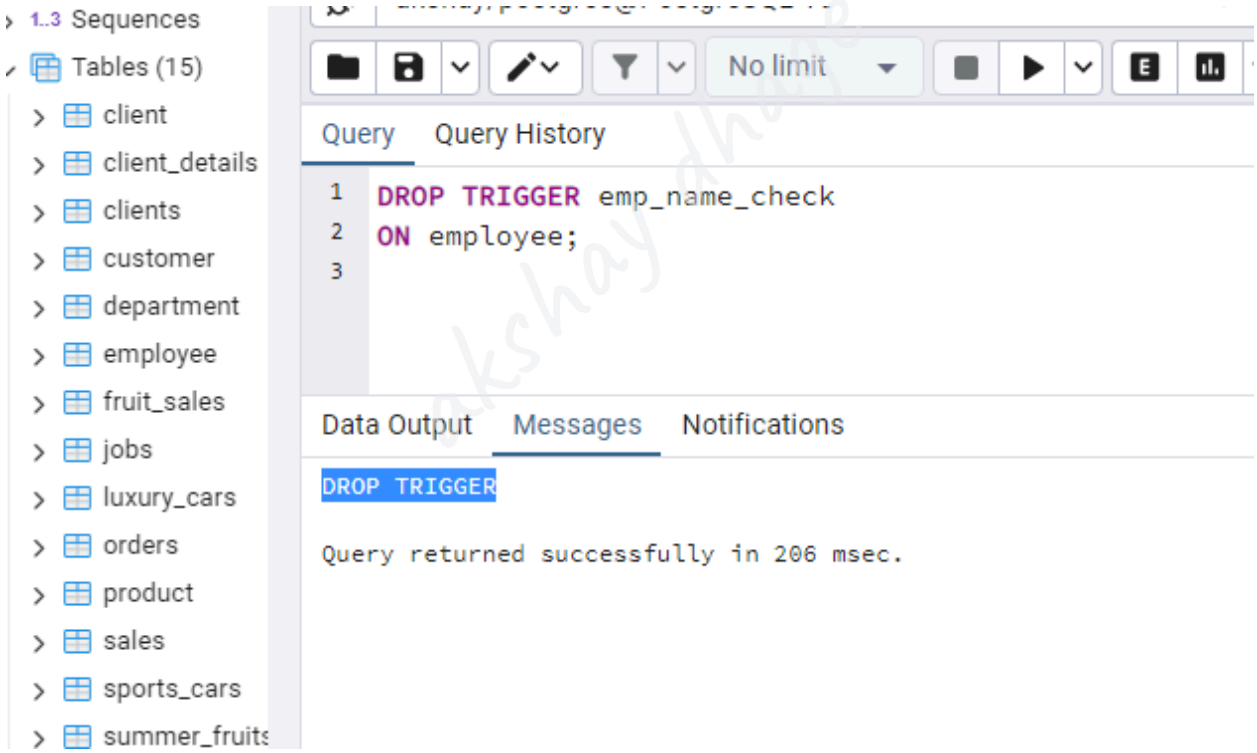
Step3: Dropping a trigger

Once the **function and trigger** have been generated successfully, we will remove the **emp_name_check** trigger with the help of the **DROP TRIGGER** command, as shown below:

```
DROP TRIGGER emp_name_check
ON employee;
```

Output

After implementing the above command, we will get the below output, which displays that the particular trigger has been **dropped** successfully from the ***Employee*** table.



ALTER TRIGGER

In this section, we are going to understand the working of the **ALTER TRIGGER** command and see the example of **altering** a trigger or rename a trigger from a specified table .

What is the ALTER TRIGGER command?

In **Trigger**, the next command is the **Alter Trigger** command, which is used to rename the existing trigger.

The syntax of PostgreSQL Alter trigger command

The following illustration is used to alter a trigger from a table:

```
ALTER TRIGGER trigger_name

ON table_name

RENAME TO new_trigger_name;
```

In the above syntax, we have used the following parameters:

Parameters	**Description**
Trigger_name	It is used to define the trigger name that we need to rename, and it is mentioned after the **ALTER TRIGGER** keyword.

Table_name	The **table_name** parameter is used to define the table name, which is connected to the trigger. And it is used after the **ON** keyword.
New_trigger_name	It is used to specify the new name of the trigger. And it is written after the **RENAME TO** keyword.

Example of PostgreSQL ALTER TRIGGER command

Let us see a sample example to understand the working of the **PostgreSQL Alter Trigger** command.

We are creating one new table as ***Student*** with the CREATE command's help and inserting some values using the INSERT command.

Step1: Creating a new table

To create a ***Student*** into an **Organization database**, we use the **CREATE** command.

But, before creating the ***Student*** table, we will use the **DROP TABLE** command to check if a similar table is already existing in the **Organization** database or not.

```
DROP TABLE IF EXISTS Student;
```

Output

After executing the above command, we will get the following window message: The ***Student*** table does not exist.

The ***Student*** table contains various columns such as **Student_id, Student_name, Scholarship** column, where we use the **Student_id** as the **GENERATED ALWAYS AS IDENTITY constraint**.

```
CREATE TABLE Student(
  Student_id INT GENERATED ALWAYS AS IDENTITY,
  Student_name VARCHAR(50) NOT NULL,
  Scholarship decimal(11,2) not null default 0,
  PRIMARY KEY(Student_id)
);
```

Output

On executing the above command, we will get the following message: The ***Student*** table has been created successfully into the **Organization** database.

Step2: Creating a new function

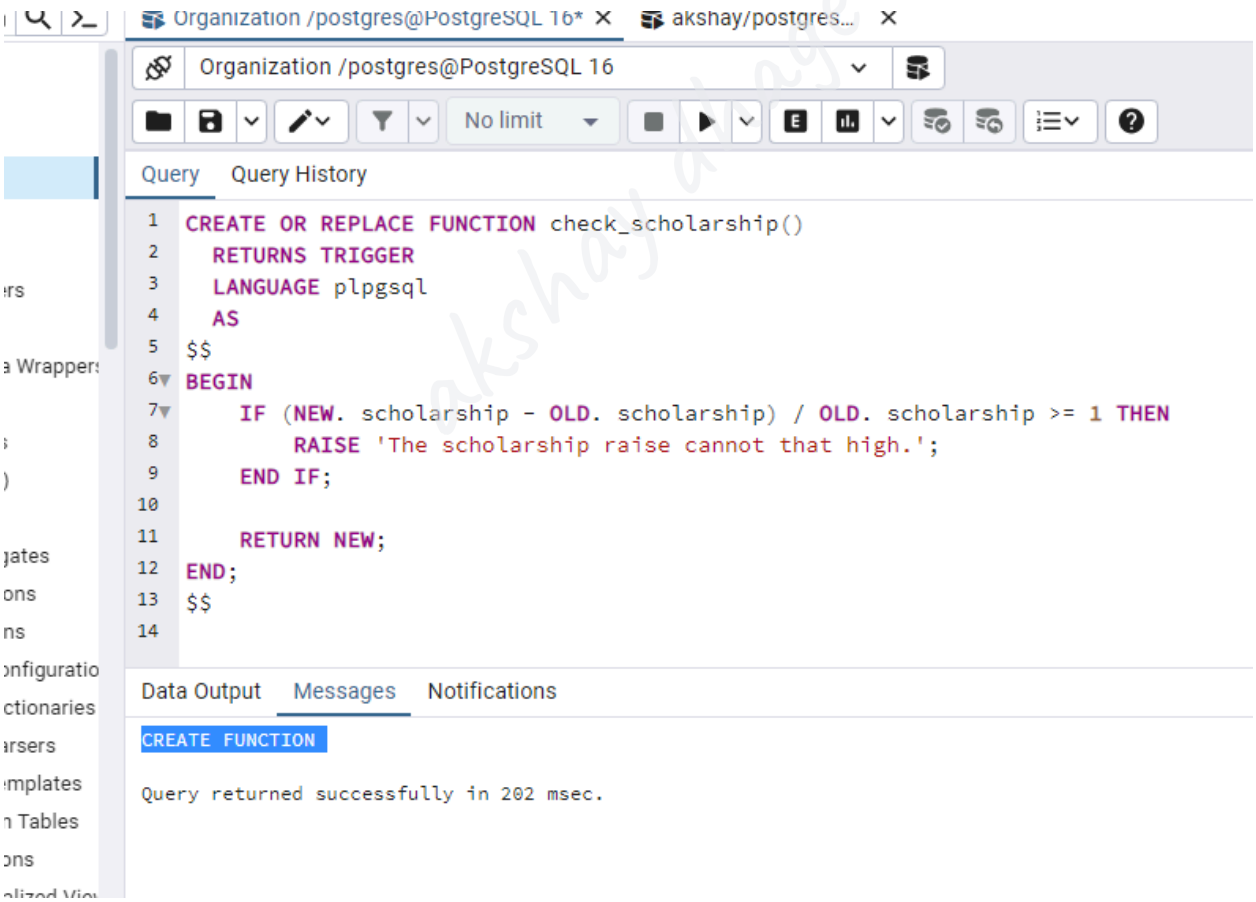
After creating the ***Student*** table successfully, we will create a new function, which raises an exception if the new scholarship is larger than the old one 100%:

```
CREATE OR REPLACE FUNCTION check_scholarship()
 RETURNS TRIGGER
 LANGUAGE plpgsql
 AS
$$
BEGIN
   IF (NEW. scholarship - OLD. scholarship) / OLD. scholarship >= 1
THEN
```

```
        RAISE 'The scholarship raise cannot that high.';
    END IF;


    RETURN NEW;
END;
$$
```

Output

On executing the above command, we will get the following message, which displays that the **check_scholarship()** function has been created successfully into the **Organization** database.

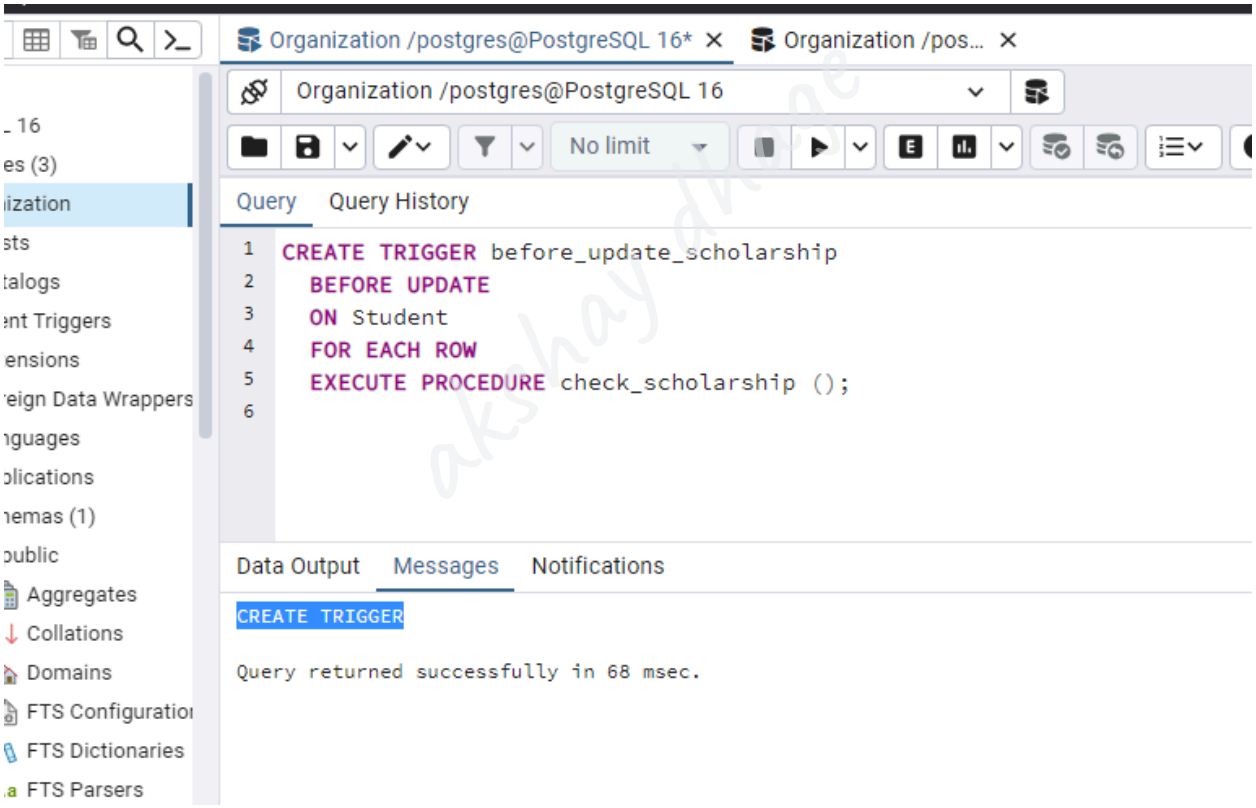


Step3: Creating a new Trigger

After creating the **check_scholarship()** function, we will **create a new trigger** on the ***Student*** table before update trigger that execute the **check_scholarship()** function before updating the scholarship.

```
CREATE TRIGGER before_update_scholarship
  BEFORE UPDATE
  ON Student
  FOR EACH ROW
  EXECUTE PROCEDURE check_scholarship ();
```

Output



Step4: Insert a new value

Once the **function and trigger** have been generated successfully, we will **insert** a new row with the **INSERT's** command help into the ***Student*** table:

```
INSERT INTO Student(Student_name, scholarship)
```

```
VALUES('Mike Ross',100000);
```

Output

After implementing the above command, we will get the below message window, displaying that the particular value has been inserted successfully into the ***Student*** table.

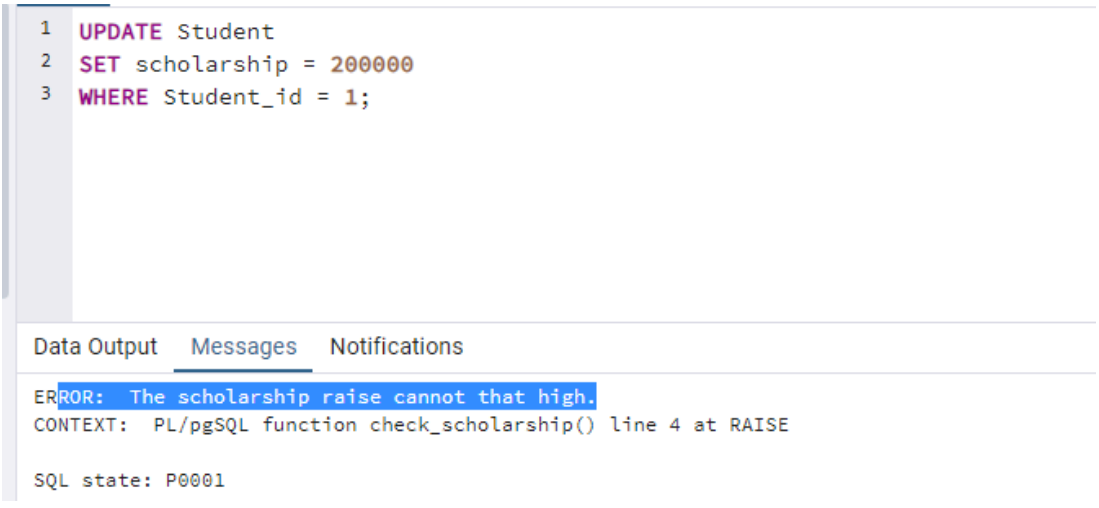
Step5: Updating the Value

After inserting the new row, we will update the scholarship of the **Student_id** 1 using the below **UPDATE** command:

```
UPDATE Student
SET scholarship = 200000
WHERE Student_id = 1;
```

Output

On implementing the above command, the trigger was executed and raise an error, which says that **the scholarship raise cannot be that high.**

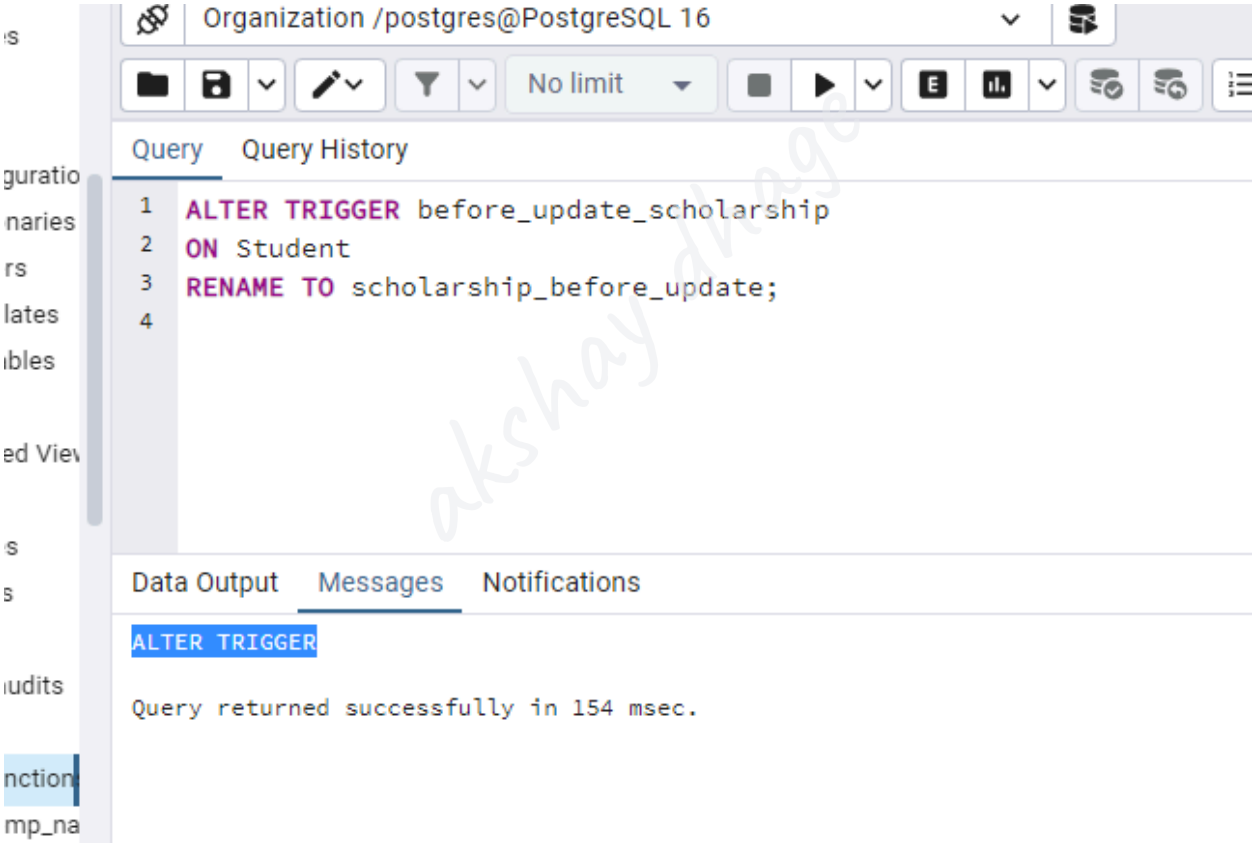


Step5: Altering the trigger command

To resolve the above error, we will use the **ALTER TRIGGER** command to rename the **before_update_scholarship** trigger to **scholarship_before_update**.

```
ALTER TRIGGER before_update_scholarship
ON Student
RENAME TO scholarship_before_update;
```

Output



Changing the triggers

In, does not contain the **OR REPLACE** command, which provides us to change the trigger explanation like the **functio**n, which will be implemented when the trigger is executed.

Therefore, we can wrap these commands in a **transaction,** and also use the **CREATE TRIGGER and DROP TRIGGER** commands.

Let us see one sample example to understand how the **DROP TRIGGER** and **CREATE TRIGGER** command works in a transaction.

The below command represents how to change the **check_scholarship()** function of the **scholarship_before_update** trigger to **validate_scholarship**:

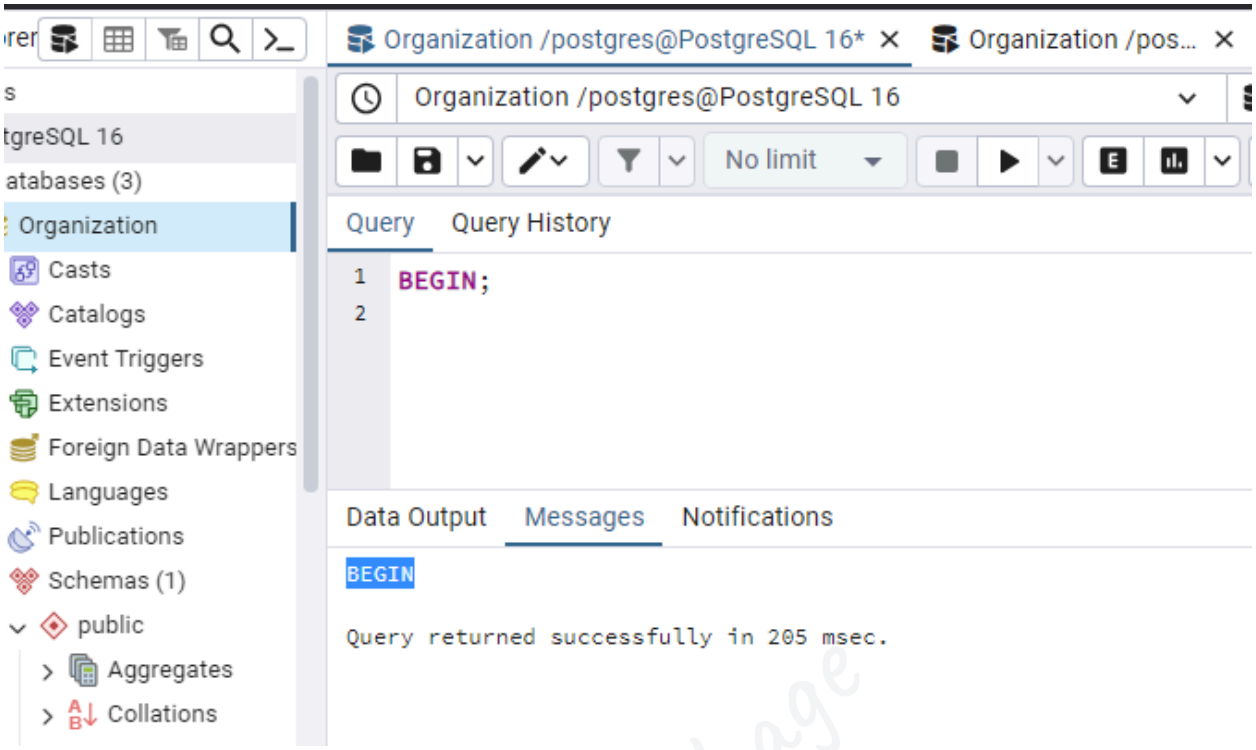
Step1: Begin a Transaction

To start a transaction, we can use the following statement:

```
BEGIN;
```

Output

After implementing the above command, we will get the below message window, which says that the specified command has been successfully implemented.



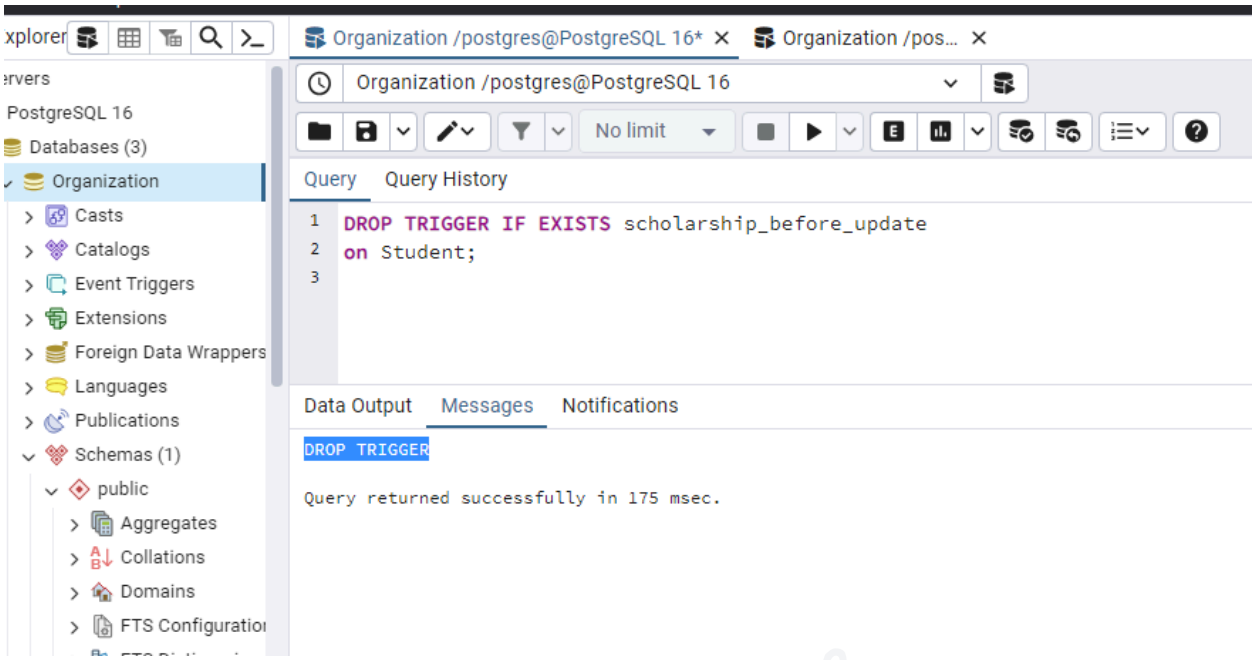
Step2: Using the DROP trigger command

After successfully starting the transaction process, we will execute the following **DROP TRIGGER** command:

```
DROP TRIGGER IF EXISTS scholarship_before_update

on Student;
```

Output

After implementing the above command, we will get the below output, which displays that the particular trigger has been dropped successfully from the ***Student*** table.



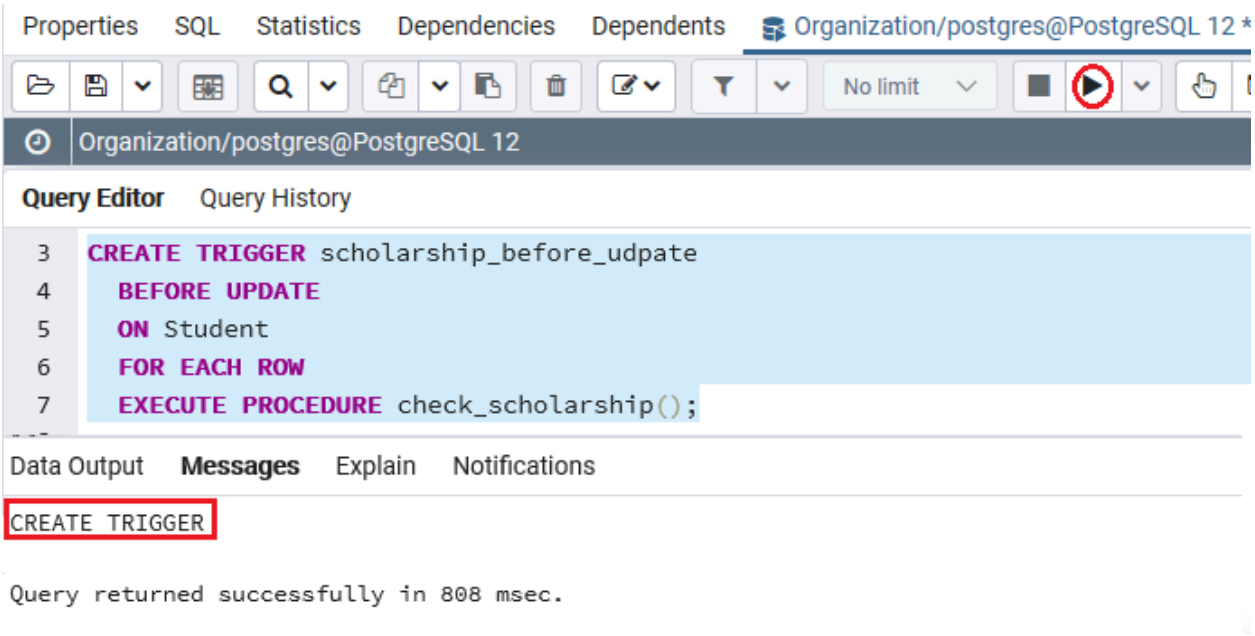
Step3: Create a new Trigger

After dropping the **scholarship_before_update trigger** successfully, we create a new trigger **scholarship_before_udpate** with a similar name, as shown in the below command:

```
CREATE TRIGGER scholarship_before_udpate
  BEFORE UPDATE
  ON Student
  FOR EACH ROW
  EXECUTE PROCEDURE check_scholarship();
```

Output

After implementing the above command, we will get the following message window, which displays that the particular trigger has been inserted successfully for the ***Student*** table.



Step4: Commit the transaction

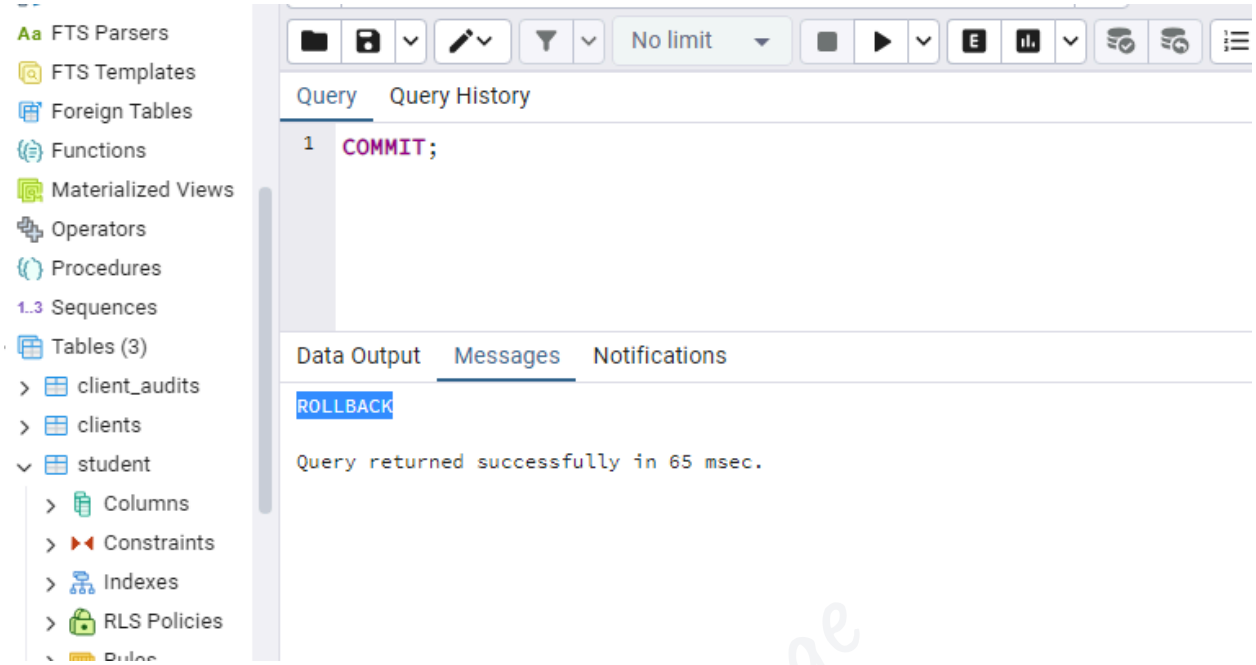
To make the change visible to other sessions (or users), we need to commit the transaction with the help of the **COMMIT** command, as shown below:

```
COMMIT;
```

Output

After implementing the above command, we will get the following message window, which displays that the transaction has been committed successfully

for the ***Student*** table.



DISABLE TRIGGER

In this section, we are going to understand the working of the **Disable triggers** using the **ALTER TABLE** command and see an **example** of it.

What is DISABLE TRIGGER command?

If we want to disable a trigger, we will use the **DISABLE TRIGGER** command with the ALTER TABLE command.

The Syntax of Disable Trigger using ALTER TRIGGER command

The syntax PostgreSQL Disable Trigger using the ALTER TRIGGER command is as follows:

```
ALTER TABLE table_name
DISABLE TRIGGER trigger_name | ALL
```

In the above syntax, we have used the following parameters, as shown in the below table:

Parameters	**Description**
Table_name	○ The **table_name** parameter is used to define the table name where the trigger is linked. ○ And it is mentioned after the **ALTER TABLE** keywords.

Trigger_name	○ It is used to define the trigger name, which we want to disable it. ○ And it can be written after **the DISABLE TRIGGER** keywords. ○ And to disable all triggers Linked with the table, we can use the **ALL** keyword as well.

Example of PostgreSQL DISABLE TRIGGER using ALTER TABLE command

Let us see a sample example to understand the working of the **PostgreSQL DISABLE Trigger** command.

- **Using Trigger name**

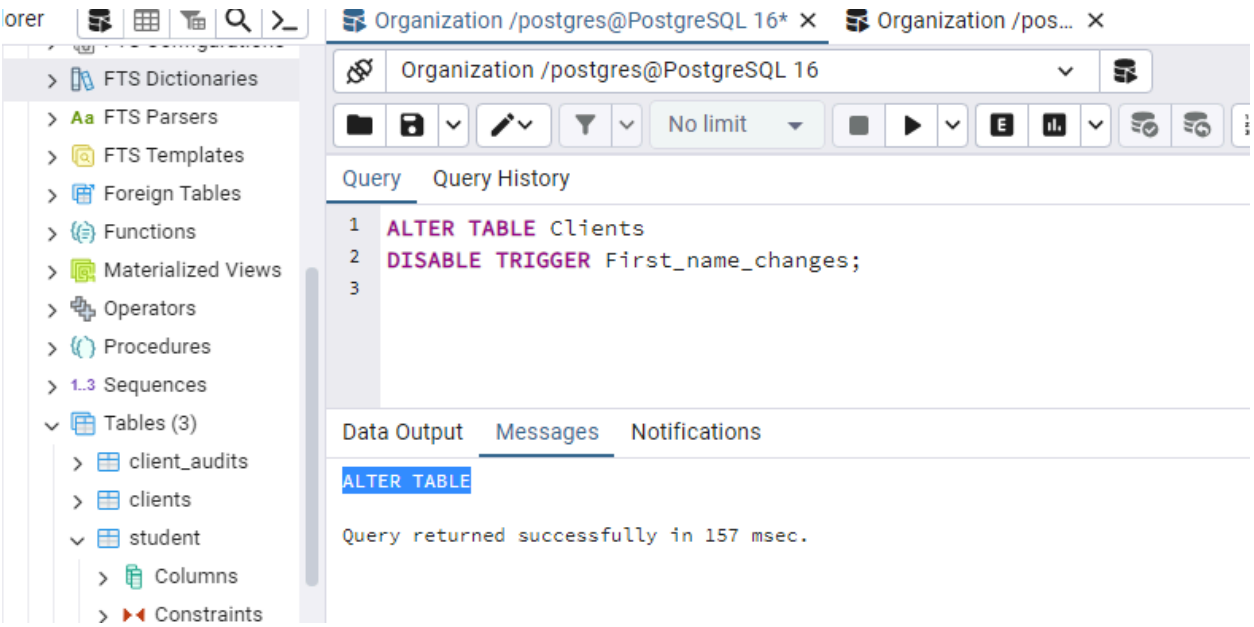
In the following example, we are taking the ***Clients*** table, which we created in the PostgreSQL Create trigger section of the PostgreSQL tutorial.

The below command disables the trigger connected with the ***Clients*** table:

```
ALTER TABLE Clients
DISABLE TRIGGER First_name_changes;
```

Output

After implementing the above command, we will get the following message window, which displays that the **First_name_changes** trigger has been **disabled** successfully into the ***Clients*** table.



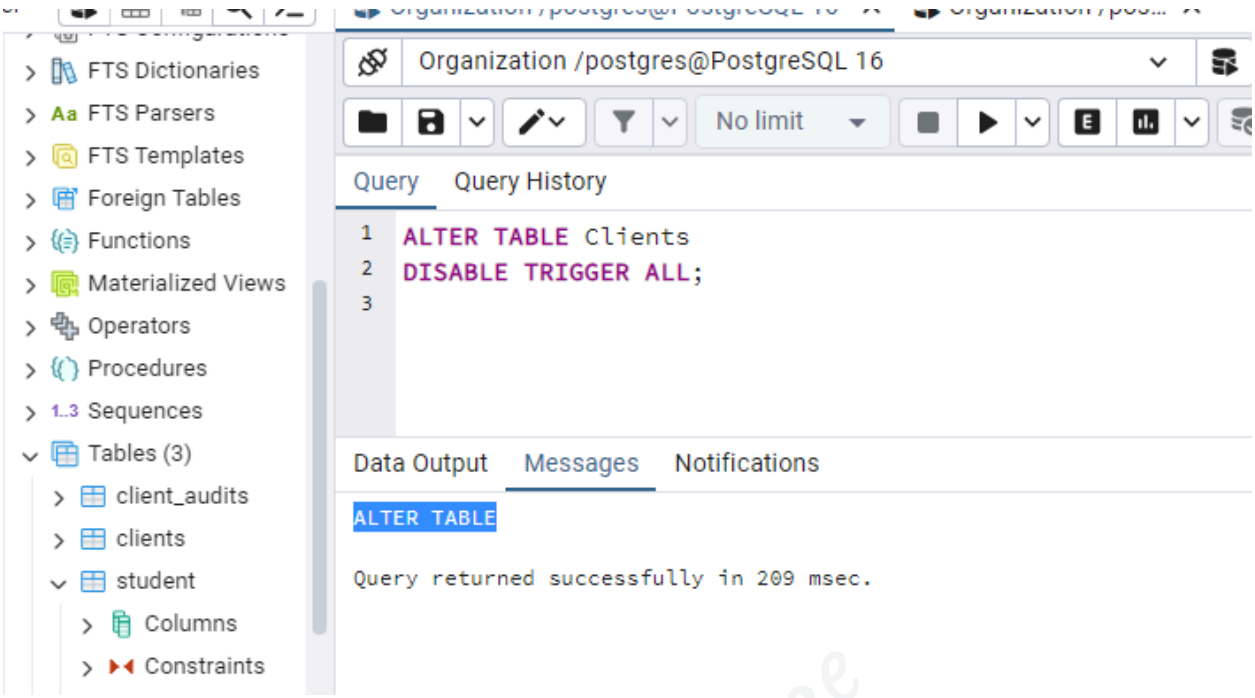
- **Using ALL keyword instead of the trigger name**

And, if we want to disable all triggers linked with the ***Clients*** table, we can use the below command:

```
ALTER TABLE Clients
DISABLE TRIGGER ALL;
```

Output

On implementing the above command, we will get the following window message, which displays that all the associated triggers with the ***Clients*** table have been disabled successfully.



ENABLE TRIGGER

In this section, we are going to understand the working of the **Enable trigger** using the **ALTER TABLE** command and see the **example** of it.

What is the ENABLE TRIGGER command?

If we want to enable a trigger, we will use the **ENABLE TRIGGER** command with the ALTER TABLE command.

The Syntax of Enable Trigger using ALTER TRIGGER command

The syntax PostgreSQL Enable Trigger using ALTER TRIGGER command is as follows:

```
ALTER TABLE table_name
ENABLE TRIGGER trigger_name | ALL
```

In the above syntax, we have used the following parameters, as shown in the below table:

Parameters	**Description**
Table_name	It is used to define the table name where the trigger is linked. And it is mentioned after the **ALTER TABLE** keywords.

Trigger_name	It is used to define the trigger name, which we want to enable it. And it can be written after **the ENABLE TRIGGER** keywords. And to enable all triggers linked with the table, we can use the **ALL** keyword as well.

Example of ENABLE TRIGGER using ALTER TABLE command

Let us see a simple example to understand the working of the **PostgreSQL ENABLE Trigger** command.

- **Using Trigger name**

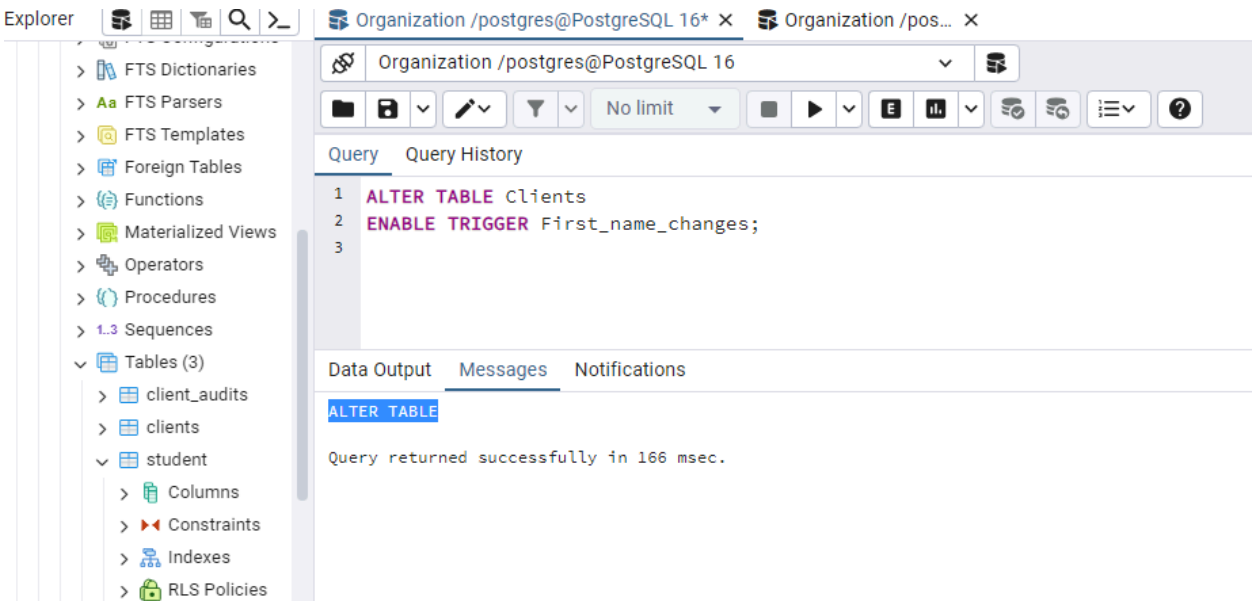
In the following example, we are taking a similar ***Clients*** table, which we used in the PostgreSQL Disable trigger section of the PostgreSQL tutorial.

If we want to enable the trigger connected with the ***Client*** table, as shown in the below command:

```
ALTER TABLE Clients
ENABLE TRIGGER First_name_changes;
```

Output

On implementing the above command, we will get the following window message, which displays that the **First_name_changes** trigger with the ***Clients*** table has been enabled successfully.



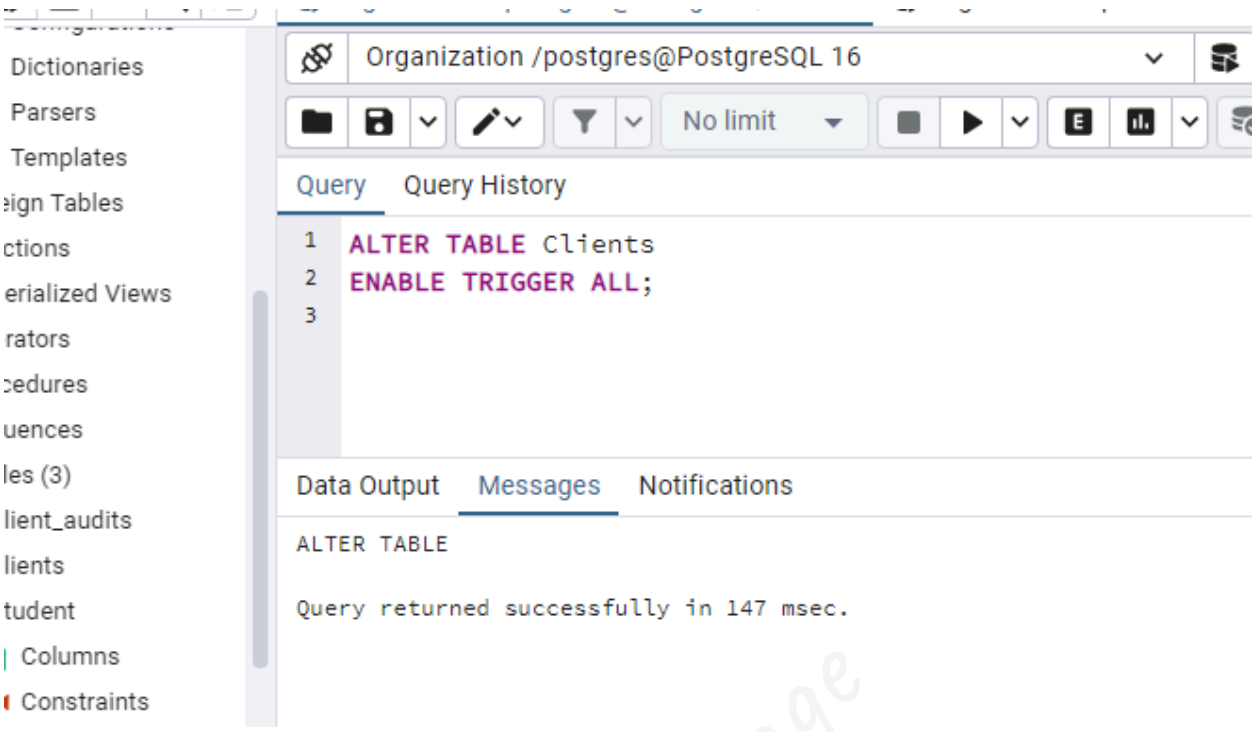
- **Using ALL keyword instead of the trigger name**

And, if we want to enable all triggers linked with the ***Clients*** table, we can use the below command:

```
ALTER TABLE Clients

ENABLE TRIGGER ALL;
```

Output

After implementing the above command, we will get the following message window, which displays that **all** the **associated** trigger has been **enabled** successfully into the ***Clients*** table.



What is Event Handling in PostgreSQL?

In PostgreSQL:

- **Event handling** means responding automatically to changes in the database.
- The "events" can be:
    - **INSERT** → new row added
    - **UPDATE** → row updated
    - **DELETE** → row deleted
- The "handler" is usually a **trigger function** that runs automatically when the event occurs.

Think of it like:

> *"When something happens in the database, do this automatically."*

Key Components

Component	Purpose
Table	Where data changes happen (e.g., employees)
Trigger Function	PL/pgSQL function that defines the action when an event occurs

Trigger	Binds the function to a table & event type (INSERT, UPDATE, DELETE)
Notification	Optional: use pg_notify + LISTEN to alert applications in real time

Step-by-Step Example: Salary Change Event

Let's build a practical example where **any salary change is recorded and notified**.

Step 1: Create Tables

```
-- Main table
CREATE TABLE employees (
    emp_id SERIAL PRIMARY KEY,
    name VARCHAR(100),
    department VARCHAR(50),
    salary NUMERIC
);


-- Audit table
CREATE TABLE salary_audit (
    audit_id SERIAL PRIMARY KEY,
    emp_id INT,
```

```
    old_salary NUMERIC,
    new_salary NUMERIC,
    changed_on TIMESTAMP DEFAULT NOW(),
    changed_by TEXT
);
```

Insert sample data:

```
INSERT INTO employees (name, department, salary)
VALUES ('Alice','Finance',55000),
       ('Bob','IT',60000),
       ('Cara','HR',50000);
```

Step 2: Create Trigger Function

```
CREATE OR REPLACE FUNCTION record_salary_change()
RETURNS TRIGGER AS $$
BEGIN
  -- Insert old and new salary into audit
  INSERT INTO salary_audit(emp_id, old_salary, new_salary, changed_by)
  VALUES (OLD.emp_id, OLD.salary, NEW.salary, current_user);

  -- Send notification to listening apps
  PERFORM pg_notify('salary_channel',
```

```
                    'Salary updated for employee ' || OLD.emp_id);


   RETURN NEW;  -- Always return NEW for UPDATE triggers

END;

$$ LANGUAGE plpgsql;
```

OLD → value before change
NEW → value after change

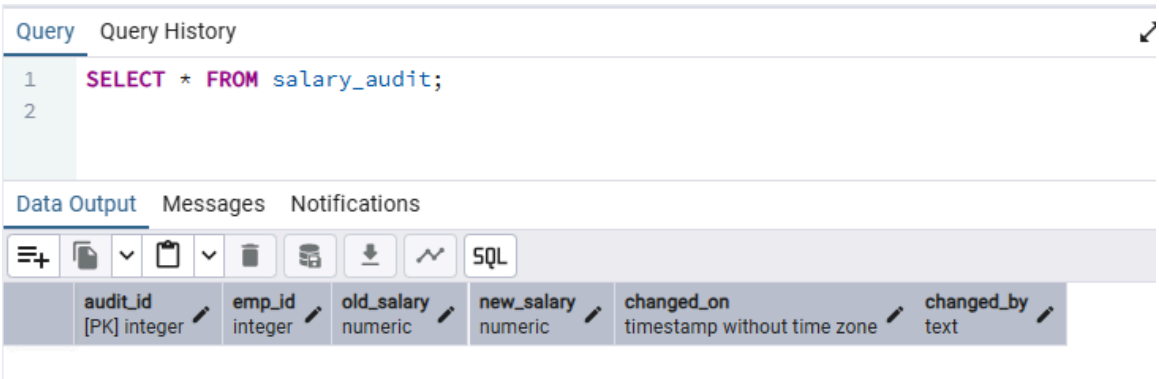
Step 3: Create Trigger

```
CREATE TRIGGER trg_salary_audit

AFTER UPDATE OF salary ON employees

FOR EACH ROW

WHEN (OLD.salary IS DISTINCT FROM NEW.salary)

EXECUTE FUNCTION record_salary_change();
```

- Fires **only when salary actually changes**.
- Runs **after the update** is committed.

Check audit table:

```
SELECT * FROM salary_audit;
```

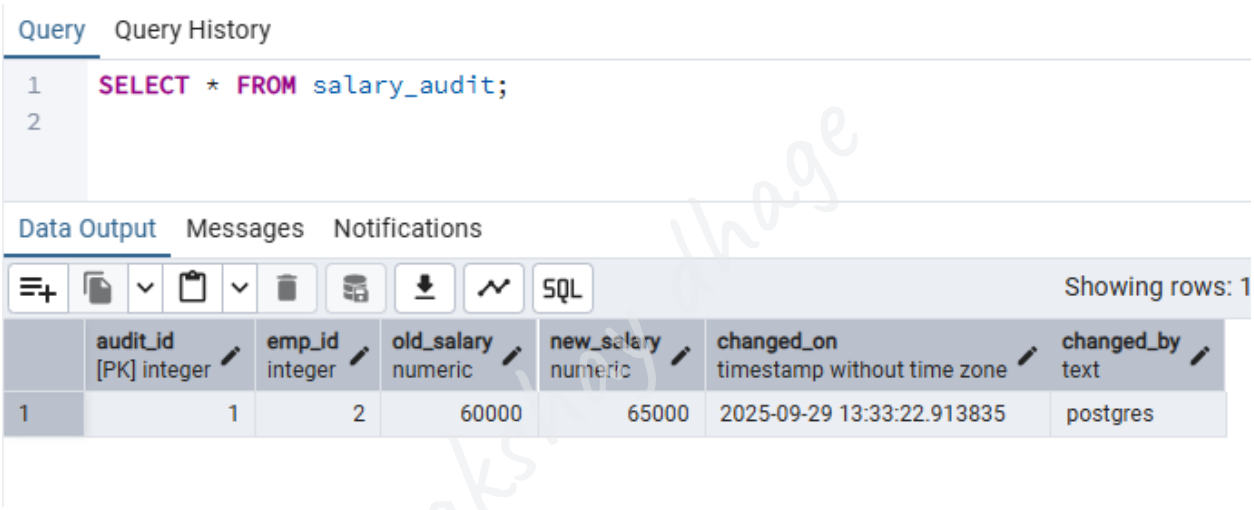


Step 4: Test It

```
UPDATE employees
SET salary = 65000
WHERE name = 'Bob';
```

Check audit table:

```
SELECT * FROM salary_audit;
```



You'll see **old & new salary logged**.

Step 5: Listen for Notifications (Optional)

```
LISTEN salary_channel;
```

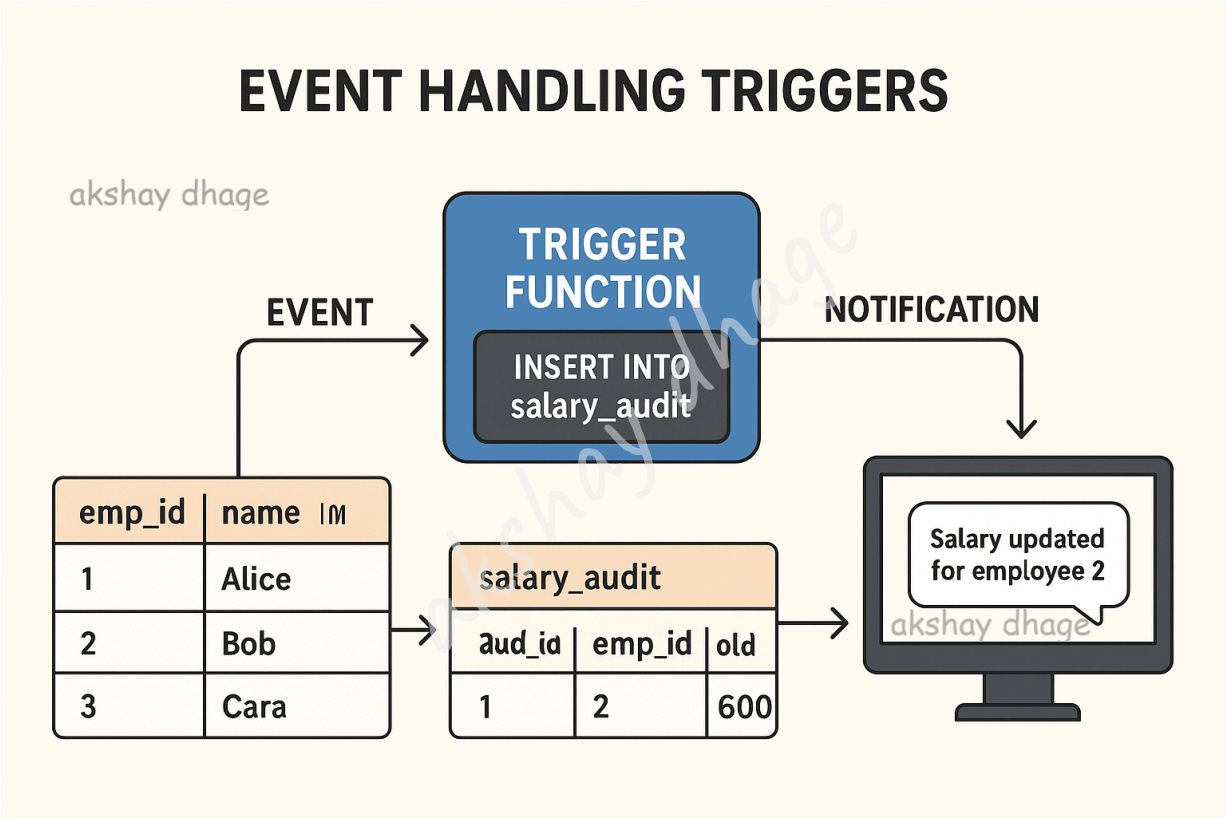
When Bob's salary changes, PostgreSQL sends:

Asynchronous notification "salary_channel" with payload "Salary updated for employee 2"\

Your app can use this to update dashboards, send emails, etc.

Recap: How Event Handling Works

1. **Event occurs** → INSERT, UPDATE, DELETE
2. **Trigger fires** → only if conditions match
3. **Trigger function executes** → logs data / notifies app
4. **External app reacts** → via LISTEN/pg_notify (optional)



Download Complete SQL & PostgreSQL Notes + Practice Files

You can access the full set of **SQL/PostgreSQL notes** along with practice datasets and queries from this GitHub repository:

👉 **SQL-resources-and-tutorials by akshay-dhage**